Set up while you wait

1. Install Git, uv, and VS Code. uv installs Python.

   `git-scm.com · docs.astral.sh/uv · code.visualstudio.com`

2. Clone the repo in a terminal:

   ```
   git clone https://github.com/joshualent/questions-api.git
   ```

3. Open the `questions-api` folder in VS Code. In VS Code's terminal, run:

   ```
   uv sync --locked
   uv run manage.py migrate
   uv run manage.py runserver
   ```

4. Open `http://127.0.0.1:8000/admin/`. You should see the Django login page.



github.com/
joshualent/
questions-api

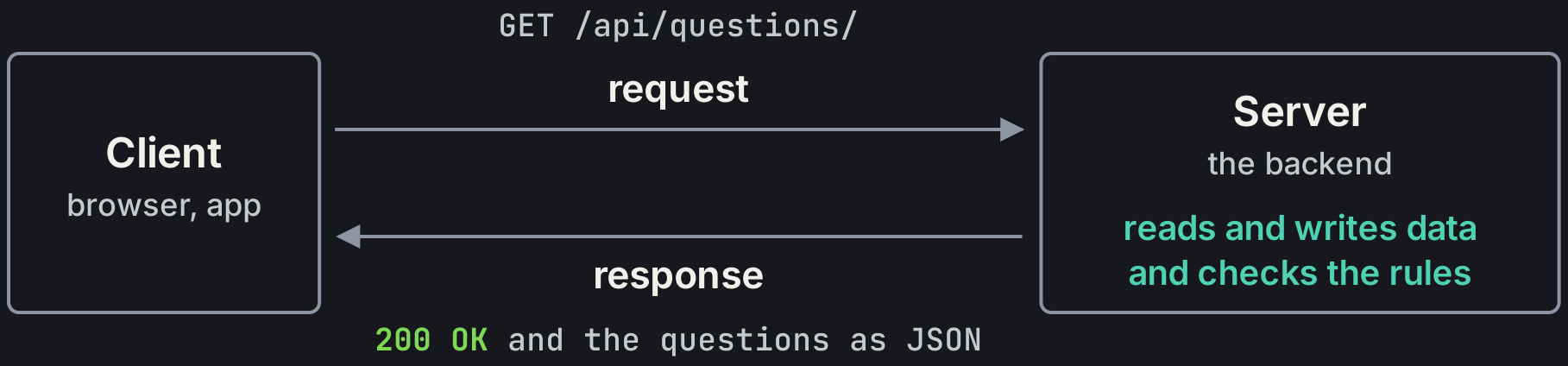
Check your setup

At `http://127.0.0.1:8000/admin/` you should see the Django login page.

Leave `runserver` running in its terminal. Open a second terminal for every other command. In VS Code, choose New Terminal from the Terminal menu.

Each part ends on a checkpoint branch with the finished code. If you fall behind, the first slide of the next part shows one line to run.

What a backend does



Server-rendered pages vs. an API

Server-rendered page

Server

Browser

Questions

What is an API?

3 votes

The server builds the whole page, with the data in it, and sends it.

An API

Server

```
[{"text": "What is an API?",
  "votes": 3, …}]
```

Browser, running the frontend app

Questions

What is an API?

3 votes

The server sends only the data, as JSON. The app fills it in.

Resources and URLs

A REST API organizes its data as resources, and each resource has URLs. Our API has one resource: questions.

`/api/questions/`	the collection of all questions
`/api/questions/1/`	the question with id 1

A URL says which data the request is about. It doesn't say what to do with the data.

HTTP methods

The method says what to do with the data at the URL.

`GET`	`/api/questions/`	list all questions
`POST`	`/api/questions/`	create a question
`GET`	`/api/questions/1/`	read question 1
`PUT`	`/api/questions/1/`	replace question 1
`PATCH`	`/api/questions/1/`	change part of question 1
`DELETE`	`/api/questions/1/`	delete question 1

PUT sends every field and replaces the question. PATCH sends only the fields that change.

Anatomy of a request

Request

```
POST /api/questions/ HTTP/1.1
Host: 127.0.0.1:8000
Content-Type: application/json

{"text": "What does a serializer do?"}
```

Response

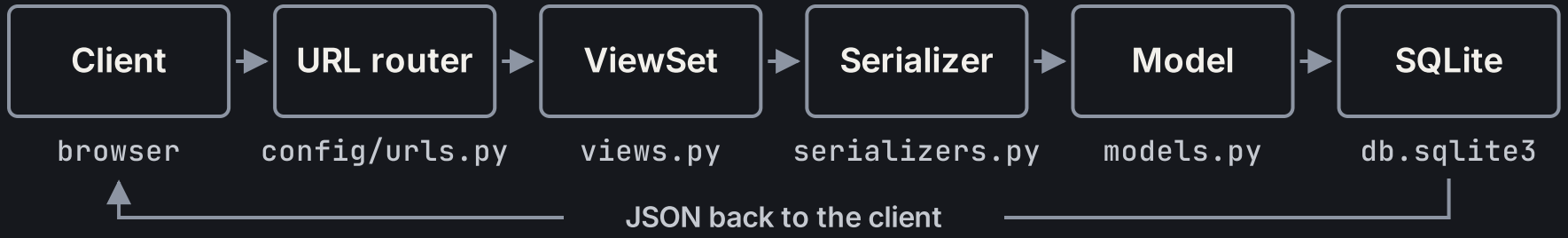
```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 1,
 "text": "What does a serializer do?",
 "votes": 0,
 "answered": false,
 "created_at": "…"}
```

`2xx` success `4xx` client error `5xx` server error

The server added id, votes, answered, and created_at.

The parts of our API



We'll build these parts from right to left, starting with the model.

Part 1: A model becomes a table

Each model becomes a database table, and each field becomes a column.

	models.py		questions_question	
text	CharField	→	varchar(300)	at most 300 characters
votes	IntegerField	→	integer	defaults to 0
answered	BooleanField	→	bool	defaults to false
created_at	DateTimeField	→	datetime	set when it's created
id	BigAutoField	→	integer	Django adds it: 1, 2, 3, ...

`makemigrations` writes a migration file. `migrate` applies it.



Life of a POST request



The request and the response

Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Part 2: How should voting work?

`votes` and `answered` are read-only, so no client can change them.

`PATCH` `/api/questions/1/` `{"votes": 9999}`

If votes were writable, any client could set any number.

`POST` `/api/questions/1/upvote/` no body

The server applies the rule: each upvote adds one.

Some operations aren't create, read, update, or delete. DRF calls them **actions**.

If you're behind, run this in your second terminal:

```
git stash -u; git switch checkpoint-1-crud; uv run manage.py migrate
```

Part 3: Who can do what

Authentication finds out who sent a request. Authorization decides whether that user may make it.

Operation	Who can do it	Permission class
List, read, or create questions	Anyone	`AllowAny`
Upvote	Logged-in users	`IsAuthenticated`
Edit, delete, or toggle answered	Staff users	`IsAdminUser`

When a check fails, DRF sends `403 Forbidden`.

If you're behind, run this in your second terminal:

```
git stash -u; git switch checkpoint-2-actions; uv run manage.py migrate
```

Part 4: One vote per user

`votes` is a number, so the server can't tell who voted.

Part 4 makes `votes` a many-to-many field. Django stores each vote as a row in a new table:

`questions_question_votes`

id	question_id	user_id
1	1	1 (mod)
2	1	2 (sam)

The migration deletes the old vote counts.

If you're behind, run this in your second terminal:

```
git stash -u; git switch checkpoint-3-auth; uv run manage.py migrate
```

Keep the JSON compatible

Clients see only the JSON. If the existing fields keep their names and types, existing clients keep working.

Before: `votes` is an integer column

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "answered": false, …}
```

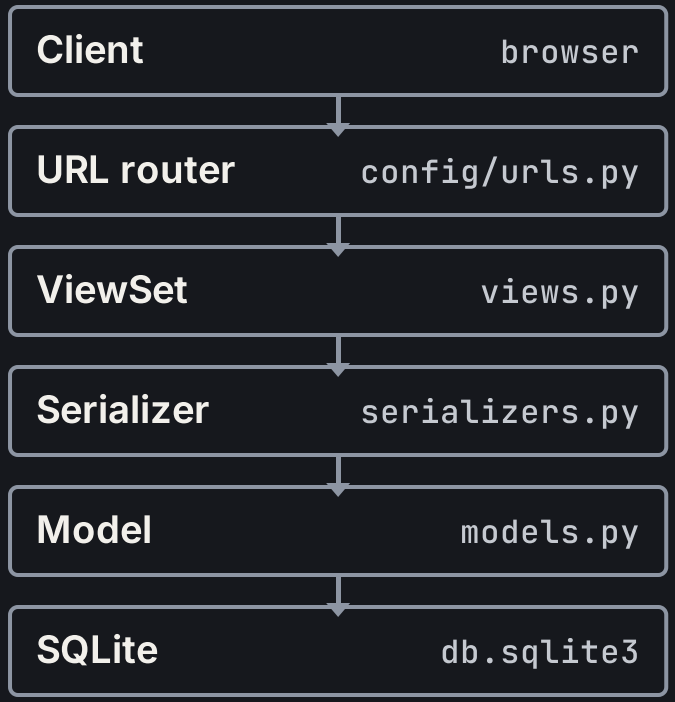
After: `votes` counts the users who voted

```
{"id": 1,
 "text": "What does a serializer do?",
 "votes": 1,
 "has_voted": false,
 "answered": false, …}
```

The serializer reports `votes` as `votes.count`, so `votes` is still a number. It also adds a new field, `has_voted`.

Renaming `upvote` to `toggle_vote` changes the URL. Clients that still call `/upvote/` get a 404.

What you built



Appendix

17 Switching checkpoints

18 Troubleshooting

19 Video still: server-rendered pages vs. an API

20 Video still: life of a POST request

21 Projector check

22 Status codes in our API

Switching checkpoints

Save your work, switch branches, and update the database:

```
git stash -u
git switch checkpoint-3-auth
uv run manage.py migrate
```

To go back from checkpoint-4-m2m, undo its migration before you switch:

```
uv run manage.py migrate questions 0001
git switch checkpoint-3-auth
```

To start fresh on any checkpoint, stop the server, delete `db.sqlite3`, and run `migrate`.

Troubleshooting

git: command not found

```
install Git from git-scm.com (macOS: run git --version)
```

uv: command not found (macOS, Linux)

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

uv: command not found (Windows PowerShell)

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 |
iex"
```

That port is already in use

```
uv run manage.py runserver 8001
```

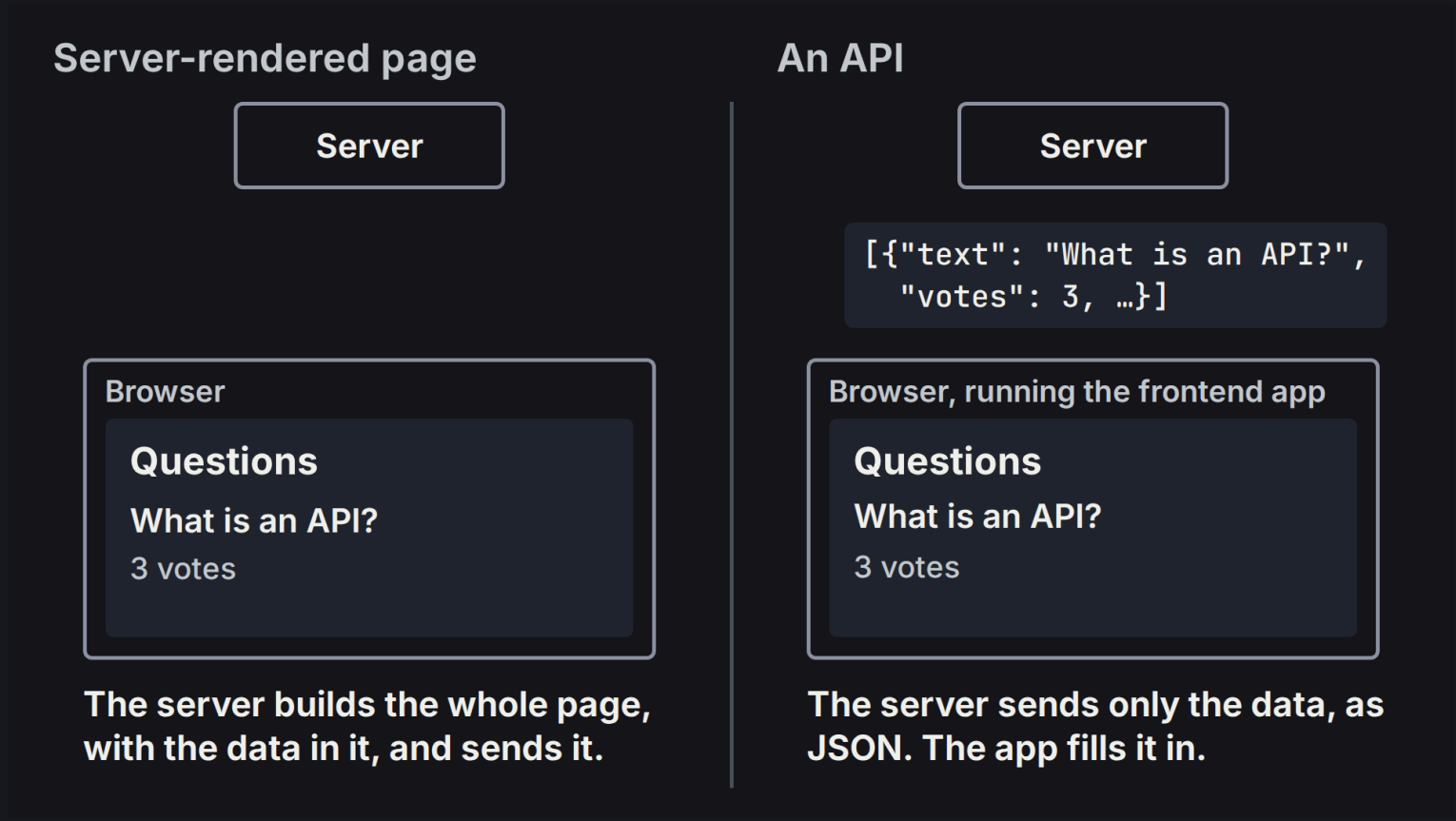
no such table

```
uv run manage.py migrate
```

No module named django

```
use uv run manage.py …, not python manage.py …
```

Video still: server-rendered pages vs. an API



Video still: life of a POST request



The request and the response

Sent `{"text": "What does a serializer do?", "votes": 9999}`

Got back `201 Created` `{"id": 1, …, "votes": 0, …}`

Blank text `400 Bad Request` `{"text": ["This field may not be blank."]}`

The client asked for 9999. The server stored 0.

Projector check

- background
- code surface
- text
- secondary text
- diagram lines
- table rules
- accent
- GET
- POST
- PUT / PATCH
- DELETE
- 2xx
- 4xx

Body text, 24 px: the smallest body size in the deck.

Secondary text, 24 px: still readable at the back?

```
Code, 18 px: question.votes.add(request.user)
```

Diagram label, 16 px: config/urls.py · serializers.py · db.sqlite3

GET POST PATCH DELETE 201 Created 403 Forbidden

```
class Question(models.Model):
    text = models.CharField(max_length=300)
    votes = models.IntegerField(default=0)
```

Status codes in our API

`200 OK`	A read, an edit, or an action worked.
`201 Created`	A POST to `/api/questions/` created a question.
`204 No Content`	A DELETE removed the question. The response has no body.
`400 Bad Request`	The data is invalid, for example blank text.
`403 Forbidden`	You aren't logged in, or you aren't allowed to make this request.
`404 Not Found`	No question has that id, or the URL doesn't exist.
`405 Method Not Allowed`	The URL doesn't accept that method, for example a GET on `/upvote/`.